

Keno API overview

When the GameWrapper is initialized, you can access the RGS features through `GameWrapper.prototype.rgs`.

The interface and the available methods depend on the `gameType` set during initialization. This document describes the methods and their responses for 'keno' games.

Note: To avoid blocked requests, it is recommended to check `GameWrapper.prototype.state` before calling RGS methods and only call them if the state is `active`. If the state is `paused`, the client should wait for a `game-resume` event before calling the desired method.

Common interfaces

RgsAccountData

```
{
  balance: number,
  currency: string
}
```

RgsCommonResponse

```
{
  game: RgsGameData,
  spin?: RgsSpinData,
  freespin?: RgsFreeSpinData,
  trigger?: RgsTriggerData
}
```

RgsGameData

```
{
  nextAction: string,
  action: string,
  totalWin?: number,
  totalBet?: number,
  ticketId?: string,
  name?: string
}
```

RgsSpinData

```
{
  s: number[],    // player selected numbers
}
```

```

d: number[], // drawn numbers
h: number[], // hits, winning numbers
tM: number, // total multiplier of all individual symbol multipliers
wA?: number, // win amount of the spin
sy?: RgsSymbolData[], // symbol data
tr?: RgsTriggerData[], // trigger data
i?: RgsInstantData // instant data
}

```

RgsFreeSpinData

```

{
s: number[], // player selected numbers
d: number[], // drawn numbers
h: number[], // hits, winning numbers
wA: number, // win amount of the spin
spR: number, // number of free spins remaining
spA: number, // number of free spins awarded
spRT: number, // number of free spins awarded during retrigger
sy?: RgsSymbolData[],
tr?: RgsTriggerData[]
}

```

RgsTriggerData

```

{
t: string, // symbol type name
o: number[], // options for selection
s: number // selected number from the options list. If s == 0 RGS selects a random o
}

```

RgsSymbolData

```

{
t: string, // symbol type name
p: number[], // position on the grid
h: number[], // hits for the symbol
m: number, // multiplier
f: boolean // true if there is a free spin feture
}

```

RgsInstantData

```

{
p: number // prize (= bet x multiplier)
}

```

Methods

- `getAccount()`
- `refresh()`
- `spin(bet: number, selectedNumbers: number[])`
- `setFreeSpin(freeSpinType: string)`
- `freeSpin(selectedNumbers: number[])`
- `close()`
- `getTicketData(ticketId?: string)`
- `setTicketData(data: string | object, ticketId?: string)`
- `getTicket(ticketId: string)`
- `sendCustomRequest(service: string, method: string, args: any[])`

getAccount()

Get user's current balance data including the currency. Calling this method will trigger a balance update throughout the whole GameWrapper.

Return value

- `Promise<RgsAccountData>` - a promise that resolves to `RgsAccountData` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see [Error Handling Guidelines](#).

Usage example

```
wrapper.rgs.getAccount()
  .then(function (data) {
    console.log('Current balance: ' + data.balance + ' ' + data.currency);
  }).catch(handleErrors);
```

Example response

```
{
  balance: 10000,
  currency: "GBP"
}
```

refresh()

Get game data and the next pending action (*spin*, *setfreSpin*, *freSpin* or *close*). `refresh()` should be the first RGS API call after GameWrapper is initialized.

Usage example

```
wrapper.rgs.refresh()
  .then(function (data) { console.log('Received game data:', data.game) })
  .catch(handleErrors);
```

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see [Error Handling Guidelines](#).

Example responses

```
{
  game: {
    nextAction: "spin",
    action: "refresh"
  }
}
```

Example response in case of an unfinished game

```
{
  game: {
    nextAction: "freespin",
    action: "refresh",
    totalWin: 0,
    totalBet: 2,
    ticketId: "fe004ae2-a32c-11e9-9bd8-4a000244a410",
    name: "enchanted-keno"
  },
  spin: {
    s: [7, 29, 36, 37, 54, 55, 59, 71, 77, 78],
    d: [2, 4, 15, 7, 9, 11, 13, 16, 18, 20, 22, 24, 6, 5, 12, 3, 27, 26, 30, 25],
    h: [7],
    wA: 0,
    sy: [
      {
        t: "C",
        p: [20, 22, 24],
        h: [20, 22, 24],
        m: 6,
        f: false
      },
      {
        t: "D",
```

```

        p: [15, 7, 9, 11, 13],
        h: [15, 7, 9, 11, 13],
        m: 1,
        f: true
    }
],
tM: 6,
tr: [
    {
        o: [20],
        s: 20,
        t: "D"
    }
]
},
fre spin: {
    s: [7, 29, 36, 37, 54, 55, 59, 71, 77, 78],
    d: [5, 2, 77, 26, 24, 41, 47, 64, 16, 14, 30, 54, 51, 8, 23, 46, 43, 57, 72, 71],
    h: [77, 54, 71],
    wA: 0,
    sy: [
        {
            t: "D",
            p: [26, 53, 19, 39, 63],
            h: [26],
            m: 1,
            f: false
        },
        {
            t: "S",
            p: [76, 56, 68, 40, 31],
            h: null,
            m: 1,
            f: false,
            pS: [56]
        }
    ],
    tM: 1,
    spR: 17,
    spA: 20
}
}

```

Example response in case of an unfinished instant game

This is an example refresh response for an unfinished instant game. Instant outcomes (**i**) in **spin** are the ones that occurred in the base game. There are also instant outcomes in **freespin**, which are cumulative outcomes for all freespins that have been played in this ticket.

If there is a need to present all instant outcomes happened, the game client should combine instant outcomes from both **spin** and **freespin** elements. Otherwise it should present only the ones from the freespin element.

```
{
  game: {
    nextAction: "freespin",
    action: "refresh",
    totalWin: 238,
    totalBet: 2,
    ticketId: "b68f1533-0794-11ea-a845-000c2932a097",
    name: "movie-theater"
  },
  spin: {
    s: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    d: [1, 80, 79, 78, 77, 76, 75, 74, 73, 72, 71, 70, 69, 68, 67, 66, 65, 64, 63, 62],
    h: [1],
    wA: 38,
    sy: [{
      t: "K",
      p: [80, 70, 69, 68, 67],
      h: [80, 70, 69, 68, 67],
      m: 1,
      f: true
    }],
    tM: 1,
    tr: [{
      o: [3],
      s: 3,
      t: "K"
    }],
    i: [
      { p: 10 },
      { p: 18 },
      { p: 4 },
      { p: 6 }
    ]
  },
  freespin: {
    s: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
```

```

    wA: 90,
    sy: [{
      t: "K",
      p: null, h: null, m: 1,
      f: false
    }],
    tM: 1,
    spR: 3,
    spA: 7,
    spRT: 1,
    i: [
      { p: 30 },
      { p: 20 },
      { p: 20 },
      { p: 20 }
    ]
  }
}

```

spin(bet: number, selectedNumbers: number[])

Purchase and start a new spin.

Arguments

- bet - {number} bet amount
- selectedNumbers - {number[]} an array of keno numbers selected by the player

Return value

- Promise<RgsCommonResponse> - a promise that resolves to RgsCommonResponse once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```

wrapper.rgs.spin(1, [5, 13, 25, 26, 33, 54, 62, 68, 69, 71])
  .then(function (data) {
    console.log('Spin done. Next action is ' + data.game.nextAction);
  }).catch(handleErrors);

```

Example response

```

{
  game: {
    nextAction: "close",

```

```

    action: "spin",
    totalWin: 4,
    totalBet: 2,
    ticketId: "aa8f5702-a3e2-11e9-b1b1-4a000244a410",
    name: "blue-cove"
  },
  spin: {
    s: [7, 29, 36, 37, 54, 55, 59, 71, 77, 78],
    d: [28, 3, 78, 74, 25, 22, 19, 31, 29, 26, 23, 48, 44, 1, 64, 61, 77, 16, 14, 36],
    h: [78, 29, 77, 36],
    wA: 4,
    sy: [
      {
        t: "R",
        p: [22, 23, 24, 32, 33, 34],
        h: [22, 23],
        m: 1,
        f: false
      },
      {
        t: "Y",
        p: [9, 10, 19, 20],
        h: [19],
        m: 1,
        f: false
      },
      {
        t: "BW",
        p: [16, 17, 26, 27, 28],
        h: [16, 26, 28],
        m: 2,
        f: false
      }
    ],
    tM: 2
  }
}

```

Example response with free spins

Example spin response when a free spin is won.

In this example symbol types ‘*R*’ and ‘*BW*’ awarded free spins.

In the trigger data list (**tr**), elements have their selected number property (**s**) set to 0. This means that **rgs.setFreeSpin()** should be called once for each of these symbol types. The RGS will then randomly determine the number of free

spins.

```
{
  game: {
    nextAction: "setfreespin",
    action: "spin",
    totalWin: 0,
    totalBet: 2,
    ticketId: "28b9d3b6-a3e6-11e9-b618-4a000244a410",
    name: "blue-cove"
  },
  spin: {
    s: [7, 29, 36, 37, 54, 55, 59, 71, 77, 78],
    d: [2, 4, 15, 7, 9, 11, 13, 16, 18, 20, 22, 24, 6, 5, 12, 3, 27, 26, 30, 25],
    h: [7],
    wA: 0,
    sy: [{
      t: "R",
      p: [2, 3, 4, 12, 13, 14],
      h: [2, 3, 4, 12, 13],
      m: 1,
      f: true
    }],
    {
      t: "Y",
      p: [46, 47, 56, 57],
      h: null, m: 1,
      f: false
    },
    {
      t: "BW",
      p: [15, 16, 25, 26, 27],
      h: [15, 16, 25, 26, 27],
      m: 1,
      f: true
    }
  ],
  tM: 1,
  tr: [{
    t: "R",
    o: [8, 9, 10, 11, 12],
    s: 0
  }],
  {
    t: "BW",
    o: [15, 17, 20, 25, 30],
    s: 0
  }
}
```

```

    }]
  },
  trigger: [{
    t: "R",
    o: [8, 9, 10, 11, 12],
    s: 0
  },
  {
    t: "BW",
    o: [15, 17, 20, 25, 30],
    s: 0
  }]
}

```

Example response with free spins and instant wins

Example spin response when a free spin with instant outcomes is won.

Instant outcomes are provided in the `i` member of the `spins` structure.

There is no need to call `rgs.setFreeSpin()` in this case - `rgs.freeSpin()` can be called immediately.

Depending on the game, the instant outcomes should either be immediately presented, or hidden and then presented during or after finishing freespins.

```

{
  game: {
    nextAction: "freespins",
    action: "spin",
    totalWin: 38,
    totalBet: 2,
    ticketId: "b68f1533-0794-11ea-a845-000c2932a097",
    name: "movie-theater"
  },
  spin: {
    s: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    d: [1, 80, 79, 78, 77, 76, 75, 74, 73, 72, 71, 70, 69, 68, 67, 66, 65, 64, 63, 62],
    h: [1],
    wA: 38,
    sy: [{
      t: "K",
      p: [80, 70, 69, 68, 67],
      h: [80, 70, 69, 68, 67],
      m: 1,
      f: true
    }],
    tM: 1,
  }
}

```

```

    tr: [{
      t: "K",
      o: [3],
      s: 3
    }],
    i: [
      { p: 10 },
      { p: 18 },
      { p: 4 },
      { p: 6 }
    ]
  },
  fre spin: {
    wA: 0,
    spR: 3,
    spA: 3
  },
  trigger: [{
    t: "K",
    o: [3],
    s: 3
  }]
}

```

setFreeSpin(freeSpinType: string)

Set the type of the next free spin game. Used for keno games where the player is offered the choice of winning different free games.

Argument

- **freeSpinType** - {string} - the desired free spin type. RGS checks the requested type and returns an error if the type isn't valid for the current game.

Return value

- **Promise<RgsCommonResponse>** - a promise that resolves to **RgsCommonResponse** once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```

wrapper.rgs.setFreeSpin('R')
  .then(function (data) {
    console.log('Free spin set. Call freeSpin() now');
  }).catch(handleErrors);

```

Example response

```
{
  game: {
    nextAction: "setfreespinspin",
    action: "setfreespinspin",
    totalWin: 0,
    totalBet: 2,
    ticketId: "28b9d3b6-a3e6-11e9-b618-4a000244a410",
    name: "blue-cove"
  },
  spin: {
    s: [7, 29, 36, 37, 54, 55, 59, 71, 77, 78],
    d: [2, 4, 15, 7, 9, 11, 13, 16, 18, 20, 22, 24, 6, 5, 12, 3, 27, 26, 30, 25],
    h: [7],
    wA: 0,
    sy: [
      {
        t: "R",
        p: [2, 3, 4, 12, 13, 14],
        h: [2, 3, 4, 12, 13],
        m: 1,
        f: true
      },
      {
        t: "Y",
        p: [46, 47, 56, 57],
        h: null,
        m: 1,
        f: false
      },
      {
        t: "BW",
        p: [15, 16, 25, 26, 27],
        h: [15, 16, 25, 26, 27],
        m: 1,
        f: true
      }
    ],
    tM: 1,
    tr: [
      {
        o: [8, 9, 10, 11, 12],
        s: 0,
        t: "R"
      }
    ],
  },
}
```

```

        {
          o: [15, 17, 20, 25,30],
          s: 0,
          t: "BW"
        }
      ]
    },
    freespin: {wA: 0, spR: 12, spA: 12},
    trigger: [
      {
        t: "R",
        o: [8, 9, 10, 11,12],
        s: 12
      },
      {
        t: "BW",
        o: [15, 17, 20, 25,30],
        s: 0
      }
    ]
  }
}

```

freeSpin(selectedNumbers: number[])

Perform a free spin.

Argument

- **selectedNumbers** - {number[]} - an array of keno numbers selected by the player. This may be different selection from the one in the initial spin, but it must be at the same level (i.e. have the same amount of values).

Return value

- **Promise<RgsCommonResponse>** - a promise that resolves to **RgsCommonResponse** once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```

wrapper.rgs.freeSpin([5, 13, 25, 26, 33, 54, 62, 68, 69, 71])
  .then(function (data) {
    console.log('Free Spin done. Next action is ' + data.game.nextAction);
  }).catch(handleErrors);

```

Example response

```
{
  game: {
    nextAction: "freespins",
    action: "freespins",
    totalWin: 0,
    totalBet: 2,
    ticketId: "28b9d3b6-a3e6-11e9-b618-4a000244a410",
    name: "blue-cove"
  },
  freespins: {
    s: [7, 29, 36, 37, 54, 55, 59, 71, 77, 78],
    d: [79, 46, 53, 50, 67, 73, 15, 12, 38, 35, 60, 56, 4, 10, 25, 22, 45, 42, 39, 70, 5],
    h: [29],
    wA: 0,
    sy: [
      {
        t: "R",
        p: [63, 64, 65, 73, 74, 75],
        h: [73],
        m: 1,
        f: false
      },
      {
        t: "BW",
        p: [13, 14, 23, 24, 25],
        h: [25],
        m: 1,
        f: false
      },
      {
        t: "S",
        p: [38, 39, 40],
        h: [38, 39],
        m: 1,
        f: true
      }
    ],
    tM: 1,
    spR: 29,
    spA: 30,
    spRT: 1,
    tr: [
      {
        t: "S",
```

```

        o: [1],
        s: 1
    }
]
},
trigger: [
    {
        t: "S",
        o: [1],
        s: 1
    }
]
}

```

Example response when maximum win is reached

When maximum win is reached, no more free spins will be allowed. `winCapped` will be set to **true** and the `spR` property won't be present in this response (as it is equivalent to 0).

```

{
  game: {
    nextAction: "close",
    action: "freespin",
    totalWin: 30000,
    totalBet: 2,
    ticketId: "1833872c-af8e-11e9-b2c7-4a000244a410",
    name: "blue-cove"
  },
  freespin: {
    s: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    d: [1, 2, 3, 4, 5, 6, 7, 8, 73, 9, 10, 11, 69, 12, 13, 66, 14, 15, 63, 16, 80, 60, 59],
    h: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    wA: 20000,
    sy: [{
      t: "R",
      p: [66, 67, 68, 76, 77, 78],
      h: [66],
      m: 1,
      f: false
    }],
    {
      t: "Y",
      p: [25, 26, 35, 36],
      h: null,
      m: 1,
    }
  }
}

```

```

        f: false
      },
      {
        t: "BW",
        p: [62, 63, 72, 73, 74],
        h: [63, 73],
        m: 1,
        f: false
      }
    ],
    tM: 1,
    winCapped: true,
    spA: 45
  }
}

```

Example response when free spin limit is reached

When the maximum number of free spins have been played, no more free spins will be allowed and **spinsCapped** will be set to **true**.

```

{
  game: {
    nextAction: "freespins",
    action: "freespins",
    totalWin: 94,
    totalBet: 2,
    ticketId: "3f58cc2c-b1e6-11e9-8b04-4a000244a410",
    name: "blue-cove"
  },
  spin: {
    s: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    d: [1, 80, 79, 78, 77, 76, 75, 74, 73, 72, 71, 70, 69, 68, 67, 66, 65, 64, 63, 62],
    h: [1],
    wA: 0,
    sy: [{
      t: "R",
      p: [68, 69, 70, 78, 79, 80],
      h: [68, 69, 70, 78, 79, 80],
      m: 1,
    }
  ]
}

```



```

        f: true
    }],
    tM: 1,
    tr: [{
        o: [30, 35, 40, 45, 50, 60],
        s: 0,
        t: "R"
    }]
},
freespins: {
    s: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    d: [16, 36, 55, 23, 33, 77, 13, 53, 18, 59, 73, 17, 1, 4, 48, 76, 47, 29, 61, 27, 35,
    h: [1, 4],
    wA: 0,
    sy: [{
        t: "R",
        p: [33, 34, 35, 43, 44, 45],
        h: [33, 35],
        m: 1,
        f: false
    }],
    {
        t: "S",
        p: [16, 17, 18],
        h: [16, 17, 18],
        m: 1,
        f: true
    }],
    tM: 1,
    spinsCapped: true, spR: 6,
    spA: 36,
    tr: [{
        o: [4],
        s: 4,
        t: "S"
    }]
},
trigger: [{
    o: [4],
    s: 4,
    t: "S"
}]
}

```

Example response when free spin is an instant game

When the free spin feature is used for an instant game the instant property (**i**) will be present, while the positional grid parameters (**p** and **h**) will be **null**.

```
{
  game: {
    nextAction: "freespins",
    action: "freespins",
    totalWin: 238,
    totalBet: 2,
    ticketId: "b68f1533-0794-11ea-a845-000c2932a097",
    name: "movie-theater"
  },
  freespins: {
    s: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    wA: 90,
    sy: [{
      t: "K",
      p: null, h: null, m: 1,
      f: false
    }],
    tM: 1,
    spR: 3,
    spA: 7,
    spRT: 1,
    i: [
      { p: 30 },
      { p: 20 },
      { p: 20 },
      { p: 20 }
    ]
  }
}
```

close()

Finish the current game and close the ticket.

Return value

- **Promise<RgsCommonResponse>** - a promise that resolves to **RgsCommonResponse** once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see [Error Handling Guidelines](#).

Usage example

```
wrapper.rgs.close()
  .then(function (data) { console.log('Ticket closed') })
  .catch(handleErrors);
```

Example response

```
{
  game: {
    nextAction: "spin",
    action: "close",
    totalWin: 0,
    totalBet: 2,
    ticketId: "8f72da7e-a3e3-11e9-b1b1-4a000244a410",
    name: "blue-cove"
  }
}
```

getTicketData(ticketId?: string)

Get data that was associated with a certain ticket.

If `ticketId` isn't specified, the wrapper will get data associated with the current ticket.

Argument

- `ticketId` - {string} ticket ID for which to get data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was previously set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicketData('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) {
    console.log('Ticket data: ' + data);
  }).catch(handleErrors);
```

Example response

```
'{"color": "red"}'
```

setTicketData(data: string | object, ticketId?: string)

Associate data with a ticket.

If `ticketId` isn't set it will default to the current ticket ID.

Arguments

- `data` - `{string | object}` the data to be associated with a ticket. May either be a JSON string or a serializable javascript object. The object will be stringified before sending the request to RGS.
- `ticketId` - `{string}` ticket ID for which to set data. If `ticketId` isn't specified it will default to the current ticket ID.

Return value

- `Promise<string>` - a promise that resolves to a string-encoded data that was set with `setTicketData()`. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

getTicket(ticketId: string)

Get ticket data for a previously played game. This is useful for getting historical ticket data, though this is usually handled by the `GameWrapper` and there is no need for the client to call this method. For more details about history replay, see the Usage Guide.

Argument

- `ticketId` - `{string}` - ticket ID of a previously played game

Return value

- `Promise<RgsCommonResponse>` - a promise that resolves to `RgsCommonResponse` once a valid response is received. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
wrapper.rgs.getTicket('ba88e2b9-2f9f-11e9-ae8a-3417ebd6a5f0')
  .then(function (data) {
    console.log('Replaying game with ticket ID: ' + data.game.ticketId);
  }).catch(handleErrors)
```

Example response

```
{
  game: {
    nextAction: "",
```

```

        action: "",
        totalWin: 0,
        totalBet: 2,
        ticketId: "75938cc4-a3ee-11e9-b76e-4a000244a410",
        name: "blue-cove"
    },
    spin: {
        s: [7, 29, 36, 37, 54, 55, 59, 71, 77, 78],
        d: [43, 11, 39, 36, 63, 4, 28, 45, 51, 67, 21, 37, 34, 57, 6, 30, 44, 49, 1, 23],
        h: [36, 37],
        wA: 0,
        sy: [
            {
                t: "R",
                p: [21, 22, 23, 31, 32, 33],
                h: [21, 23],
                m: 1,
                f: false
            },
            {
                t: "Y",
                p: [14, 15, 24, 25],
                h: null,
                m: 1,
                f: false
            },
            {
                t: "BW",
                p: [16, 17, 26, 27, 28],
                h: [28],
                m: 1,
                f: false
            }
        ],
        tM: 1
    }
}

```

sendCustomRequest(service: string, method: string, args: any[])

Send a custom RGS request through the wrapper. This is useful when the client wants to use a feature which is supported by the RGS, but not implemented by the wrapper.

Should be used with caution. Best course of action is to ask first if this is the

right way to implement required functionality. That is to avoid unnecessary debugging and delays.

Argument

- `service` - {string} RGS service name
- `method` - {string} RGS method name
- `args` - {any[]} array of arguments for the method. May be an empty array for no arguments.

Return value

- `Promise<any>` - a promise that resolves to the response object once a valid response is received. The exact response structure depends on the RGS method that is being called. In case of failure, the promise will be rejected with an error type string. For details about error handling see Error Handling Guidelines.

Usage example

```
const gameId = /* some gameId related logic */;
wrapper.rgs.sendCustomRequest('SlotService', 'Refresh', [gameId])
  .then(function (data) { console.log('Refresh response: ', data) })
  .catch(handleErrors);
```